VS Code 的安装和 C/C++程序的运行环境配置方法

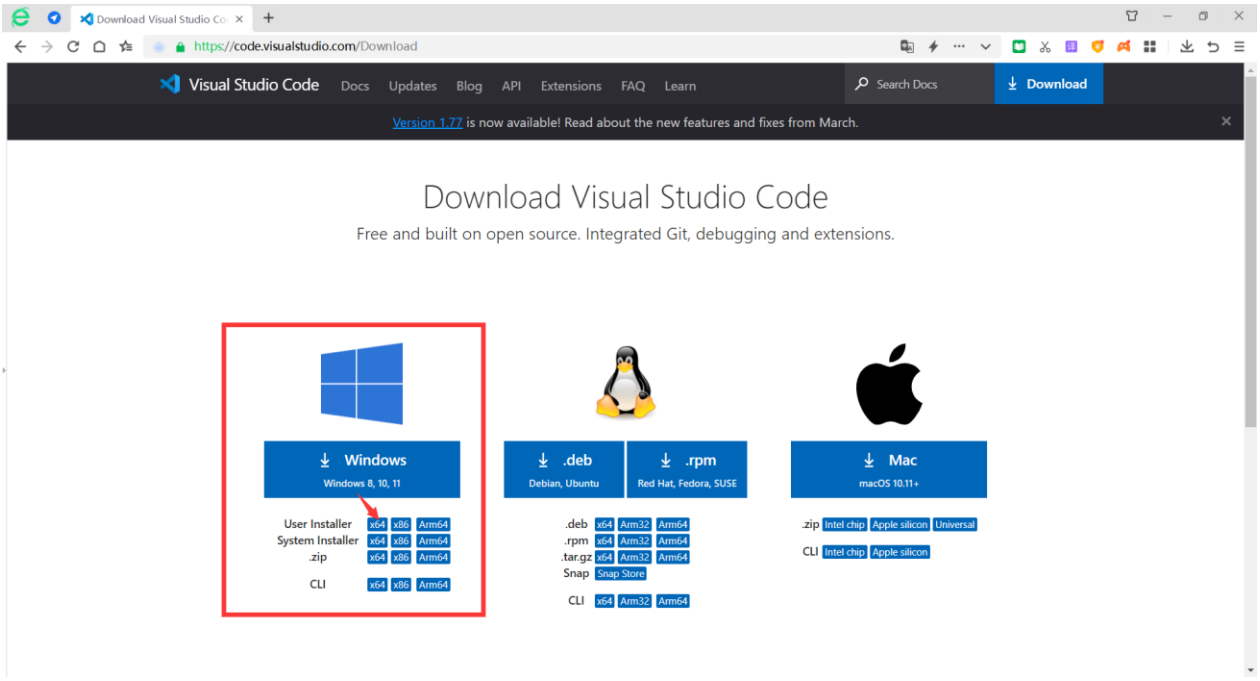
本文介绍在 Windows 操作系统中，配置 VS Code 以使用 MinGW-w64 中的 GCC 编译器(gcc)、GCC C++编译器(g++)和 GDB 调试器(gdb)的方法，并举例说明如何运行和调试 C/C++程序。有关 GCC，GDB，MinGW-w64 和 C++语言的相关知识在此不作详述，可以自行在网络上查阅。以下内容也可直接查看官网 Get Started with C++ and MinGW-w64 in Visual Studio Code https://code.visualstudio.com/docs/cpp/config-mingw。

✧ VS Code 的安装

1. 下载并安装 VS Code（全称 Visual Studio Code）

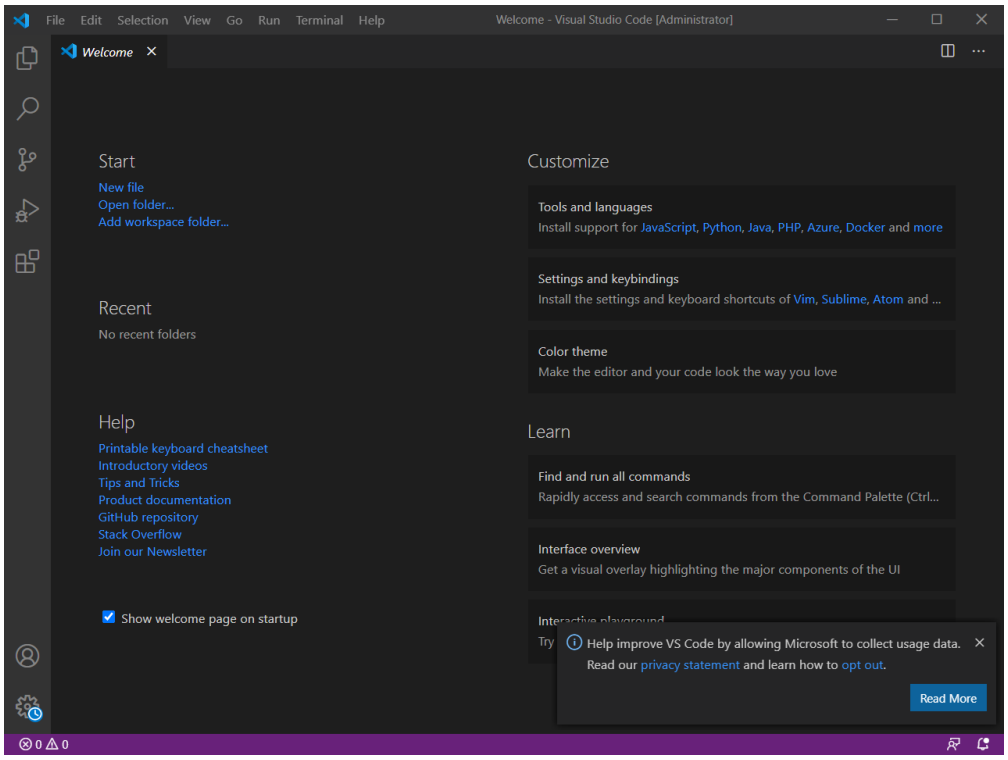
（1）下载：登录官网下载地址，根据自己电脑操作系统的类型下载对应安装程序。
https://code.visualstudio.com/download。

提示：官网下载 VS Code 很慢，建议从教学班 QQ 群下载任课教师提前下载好的安装程序。



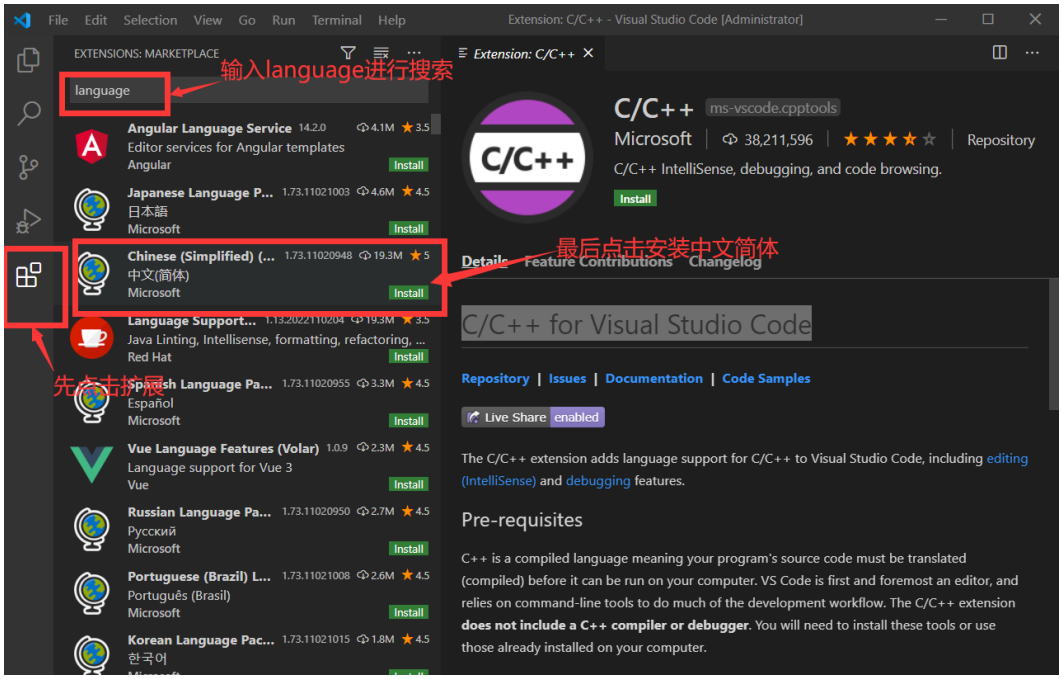


（2）安装：鼠标双击 VS Code 安装程序开始安装，注意安装路径不要有中文。安装完成后，打开 VS Code，出现欢迎界面。

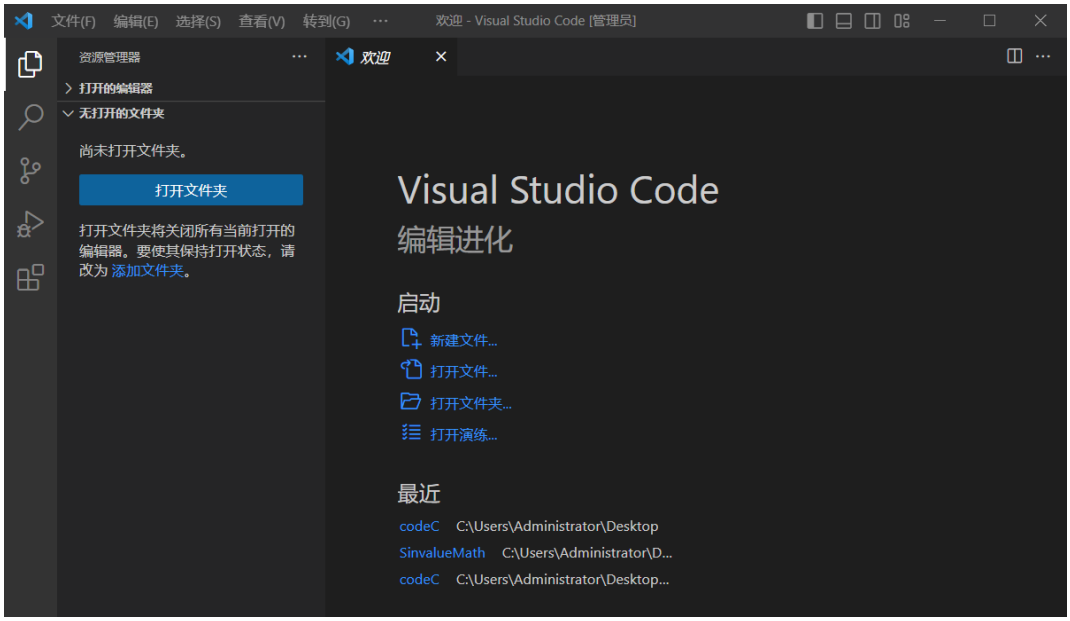


2. 中文环境的安装

中文环境通过安装扩展来实现，先点击左侧侧边栏的“extensions/扩展”，然后在搜索框中输入”language”或者”Chinese”，选择”中文(简体)”进行安装，完成后重启 VS Code 即可看到中文界面。

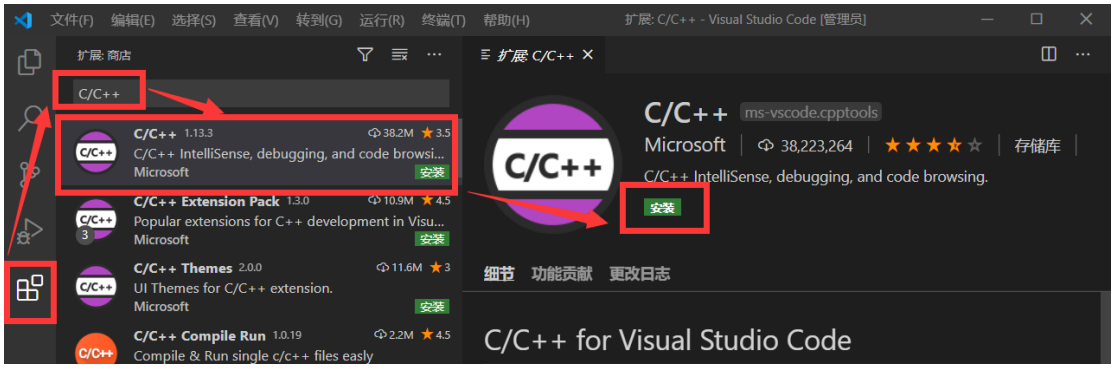


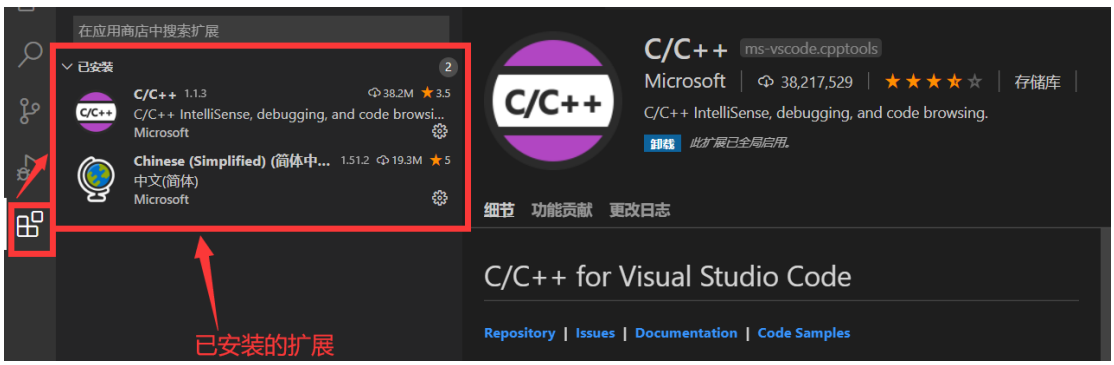
重启 VS Code 之后，用户界面变成中文版。



3. 安装 C/C++扩展插件

在 VS Code 中，点击左侧”扩展”，在搜索栏输入“C/C++”，按图示箭头顺序安装扩展。

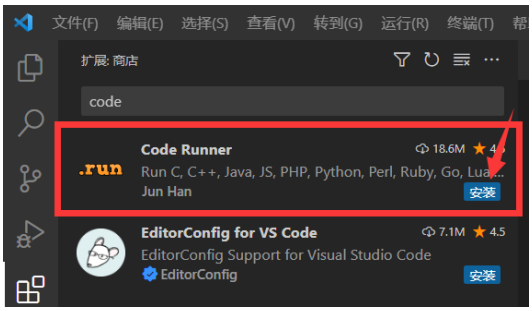




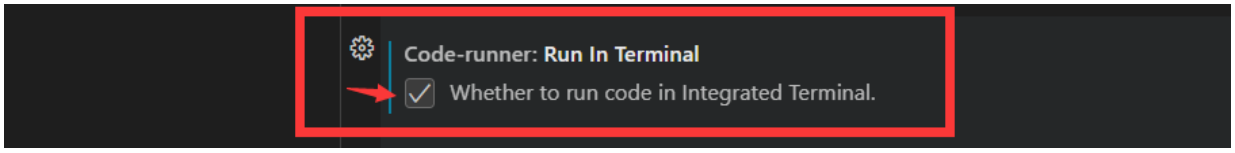
说明： VS Code 是一个编辑器，C/C++扩展插件不包括 C/C++编译器和调试器。因此，安装了 C/C++扩展插件之后可以编辑 C/C++程序，但并不能立即编译和调试程序，还需要另外安装编译或调试工具，或使用计算机上已经安装的相应工具。

4. 安装 Code Runner 扩展插件

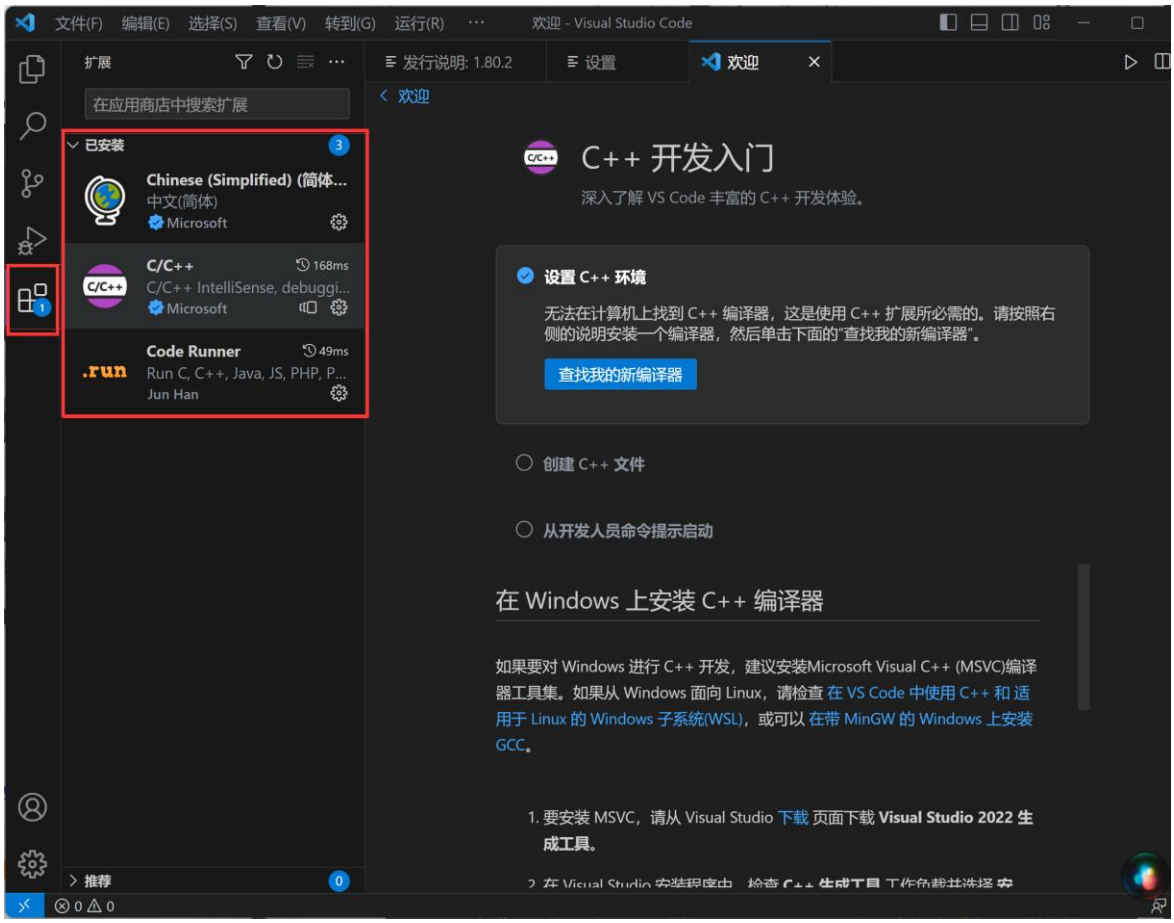
在 VS Code 中，点击左侧”扩展”，在搜索栏输入“Code Runner”，安装扩展。



然后打开“文件”菜单，选择”首选项”，点击”设置”，在搜索栏搜索 Run In Terminal，勾选图示“Whether to run code in Integrated Terminal.”。



在 VS Code 中，点击左侧”扩展”，可以看到已安装的应用。

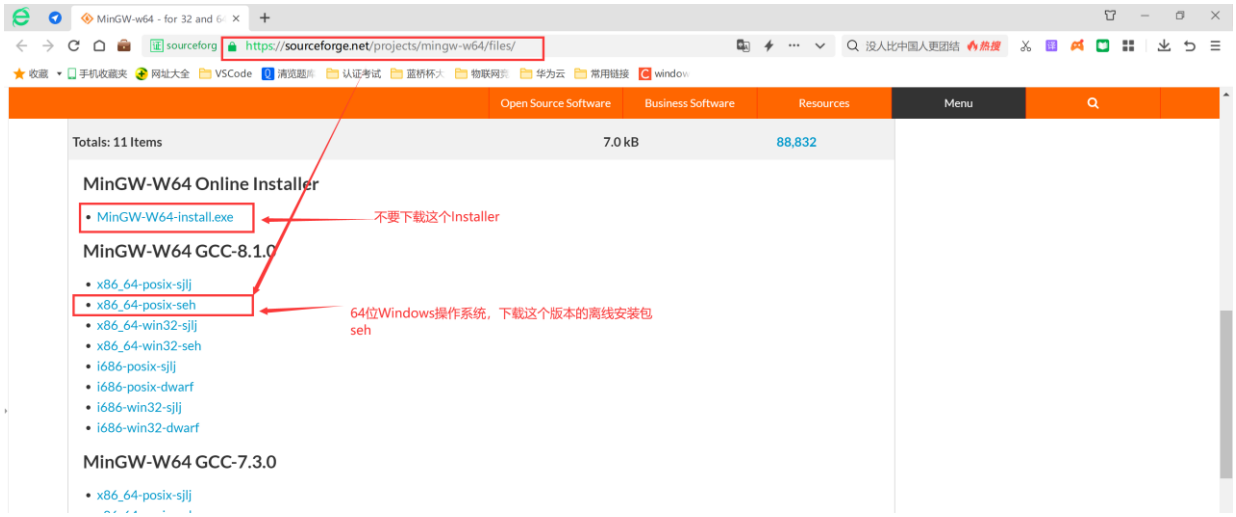


✧ 通过 MinGW 使用 GCC、G++和 GDB

1. 下载和安装 MinGW-w64 编译器。

本文介绍下载 MinGW-w64 离线安装包进行安装的方法，下载地址：
https://sourceforge.net/projects/mingw-w64/files/
图示 Installer 安装会出问题，请直接下载离线安装包。64 位 Windows 操作系统，可以下载图示版本的离线安装包。其中 seh 和 sjlj 版本，表示两个不同的异常处理系统，seh 不会调用 Windows

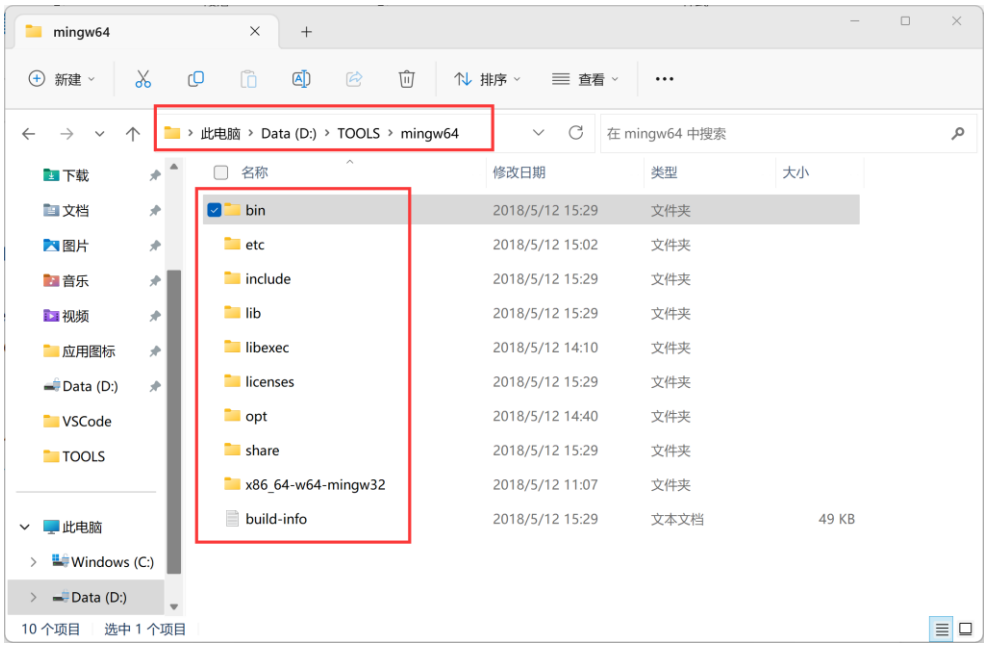
自带的异常处理，无异常处理开销，如需处理异常需要程序员自己编写异常处理的代码；sjlj 支持 Windows 的异常抛出，运行性能略有影响。



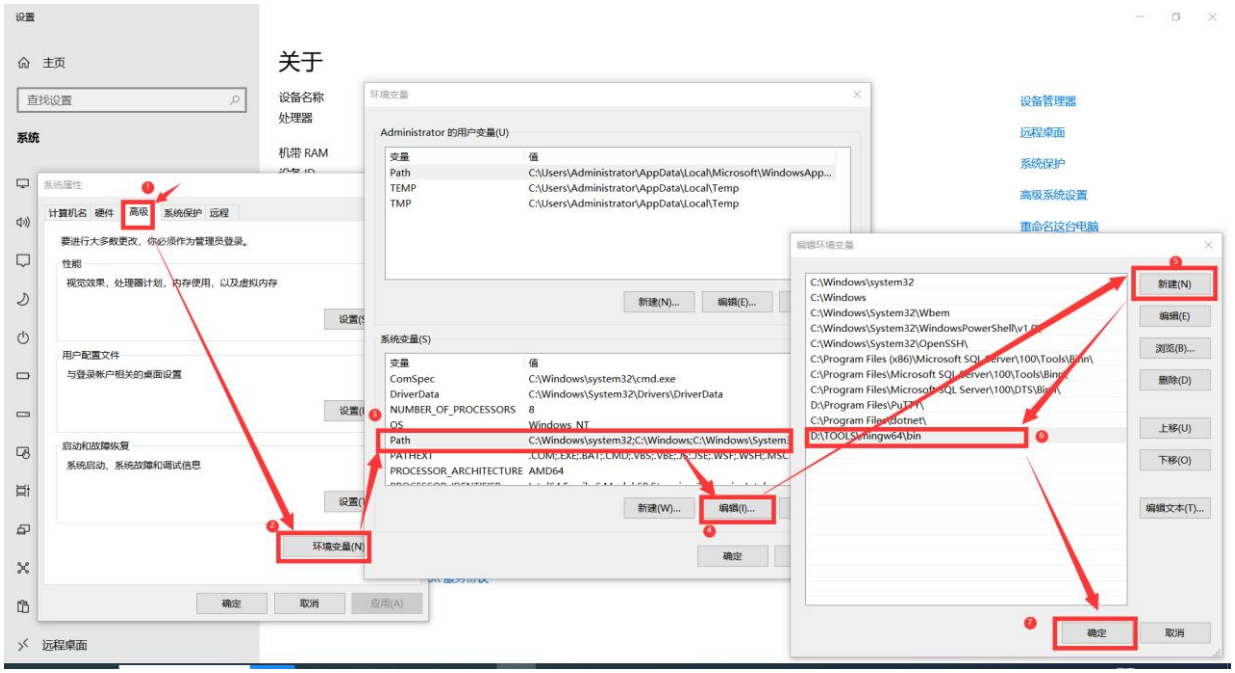


下载的安装包文件： x86_64-8.1.0-release-posix-seh-rt_v6-rev0

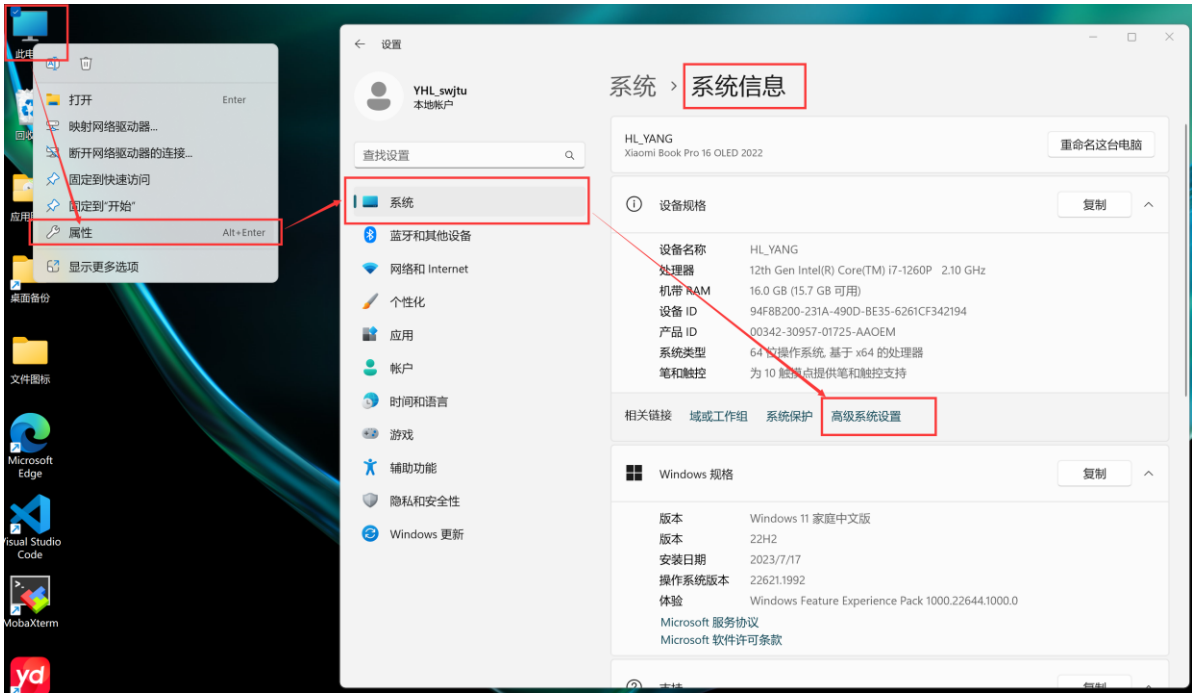
离线安装包无需安装，可直接解压到指定路径，如 D:\TOOLS\mingw64，如下图所示。



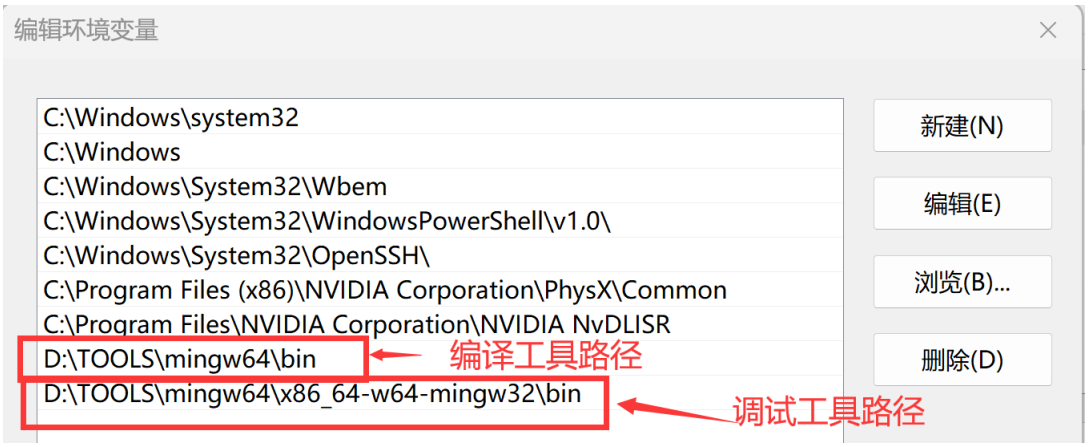
接下来，将 mingw64 文件夹中的的 bin 文件夹的路径（如：D:\TOOLS\mingw64\bin），添加到系统环境变量中。若操作系统是 Windows8.1，先在桌面“我的电脑”右击鼠标选择“属性”，然后进行环境变量设置。具体步骤如下图所示。



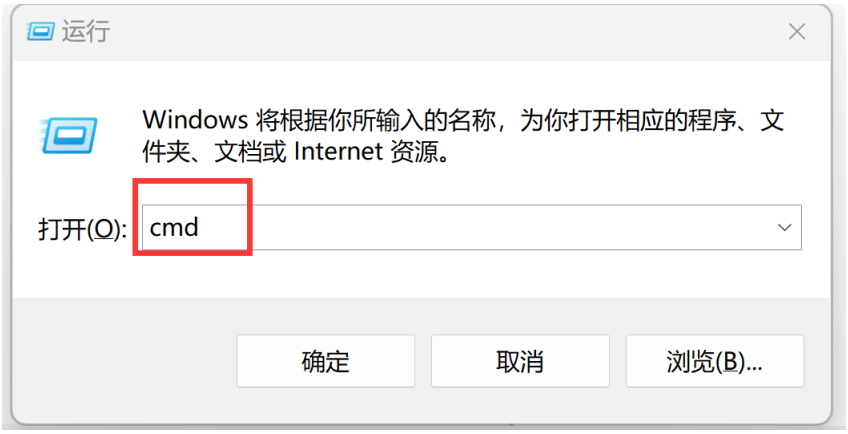
如果是 Windows11 操作系统，进行环境变量设置的前几步如下图所示，后续环境变量设置步骤与上图 Win8.1 系统一样。



按同样的方法添加调试工具的路径到系统环境变量，编写好程序后就可以进行调试。
调试工具的路径，如：D:\TOOLS\mingw64\x86_64-w64-mingw32\bin。



测试 MinGW 是否安装成功：快捷键 win+R 键运行，输入命令 cmd。

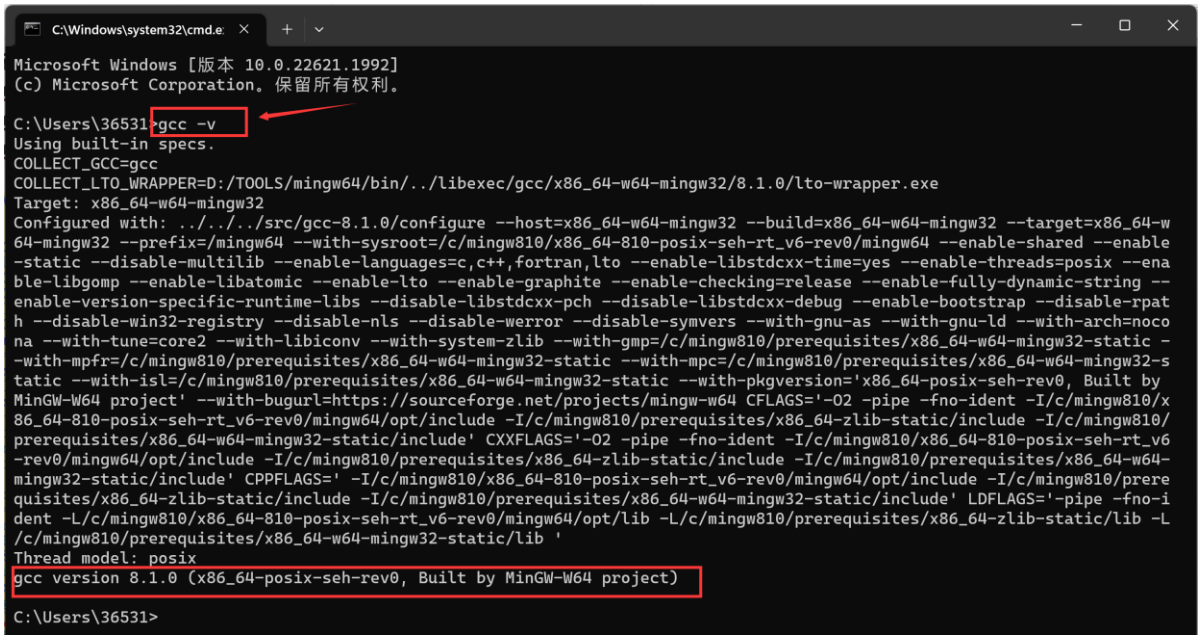


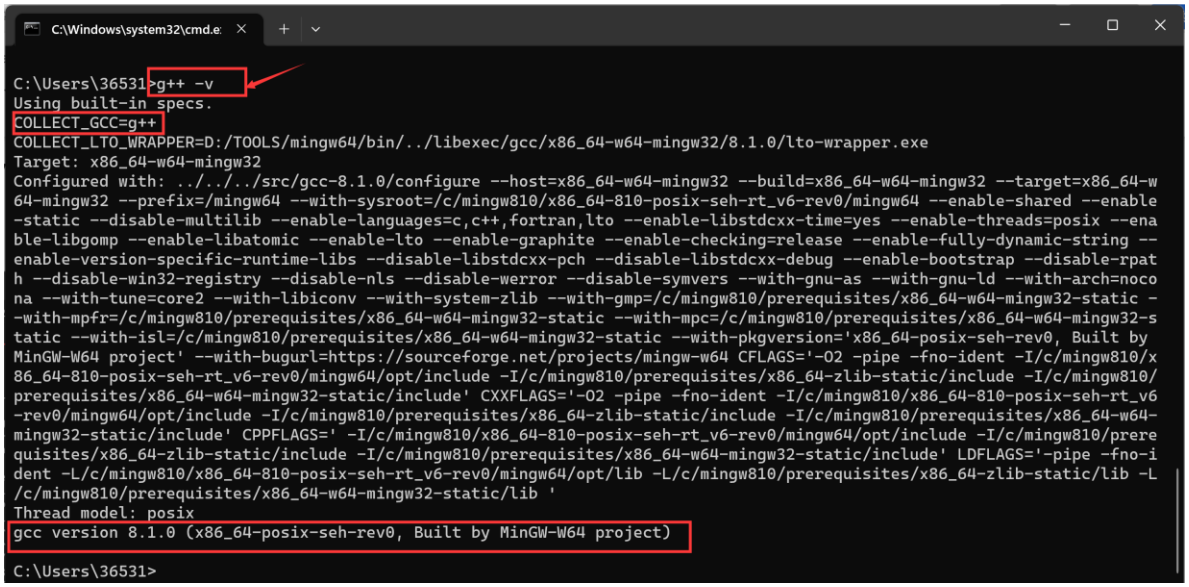
在命令行中依次输入以下命令，出现正确的版本信息则安装成功。

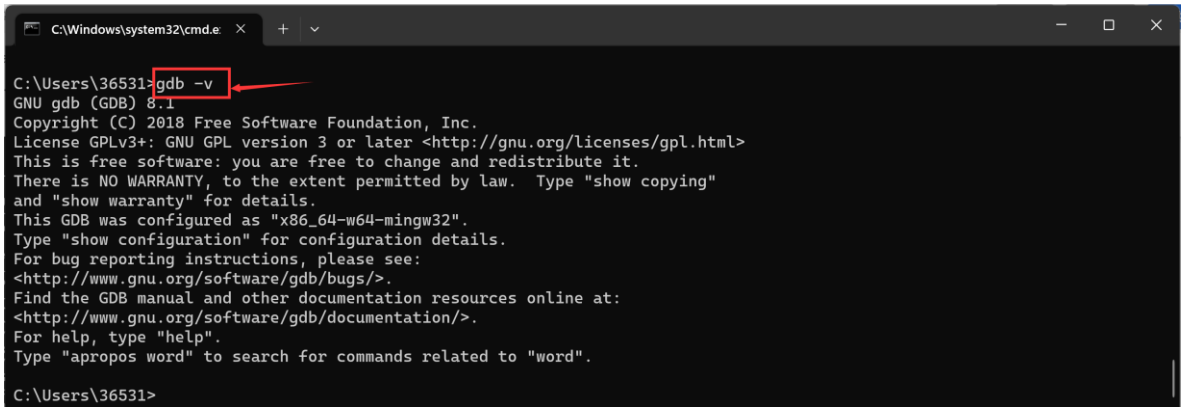
gcc -v

g++ -v

gdb -v





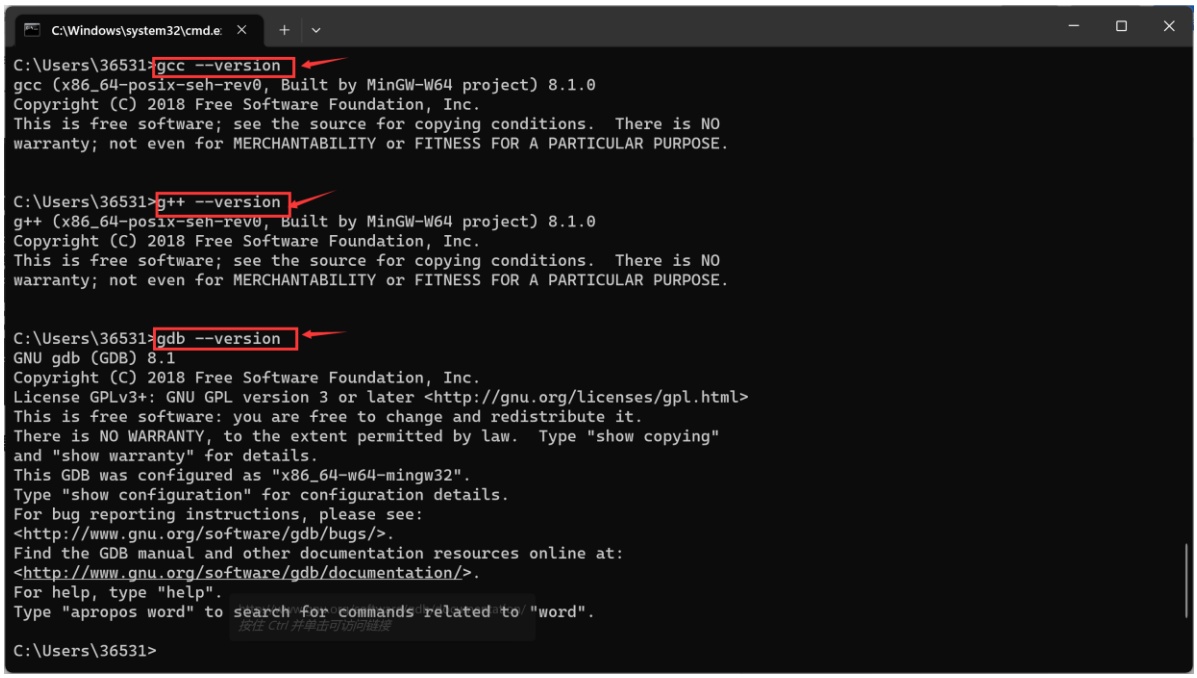


也可以输入以下命令确定是否安装成功：

gcc --version

g++ --version

gdb –version

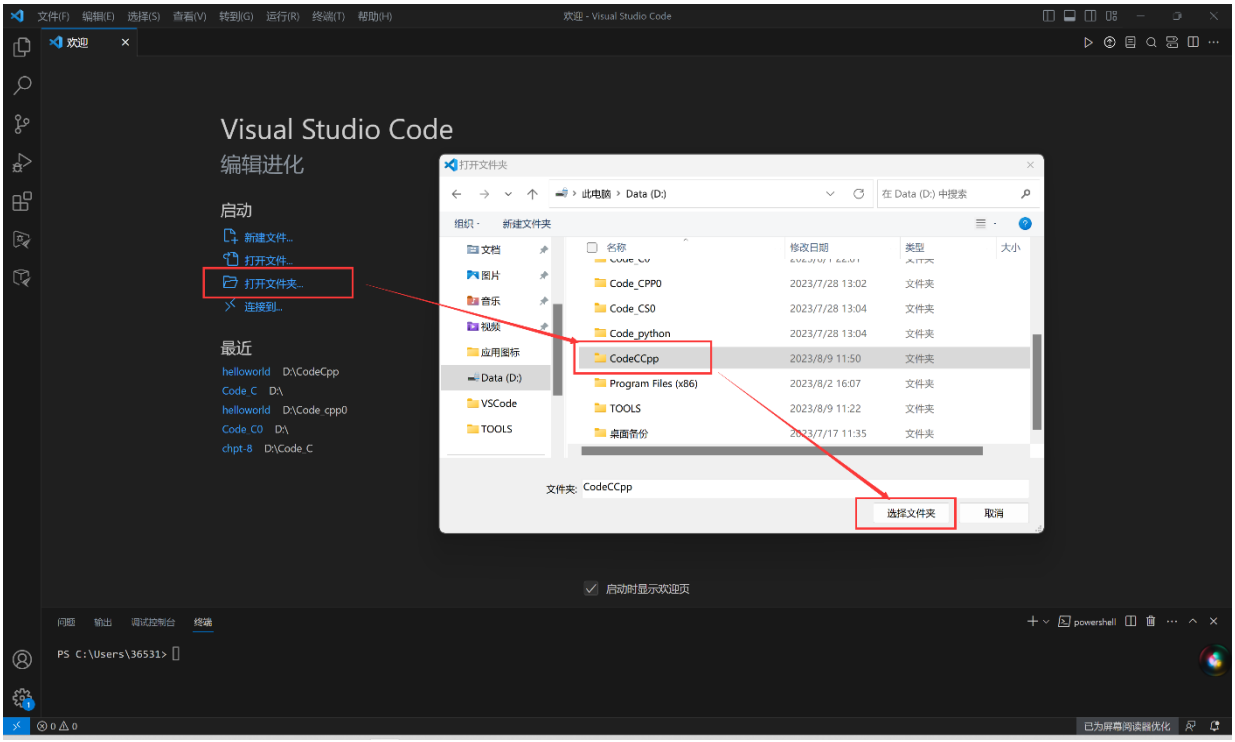


2. 使用 VS Code 编译和调试 C/C++语言程序

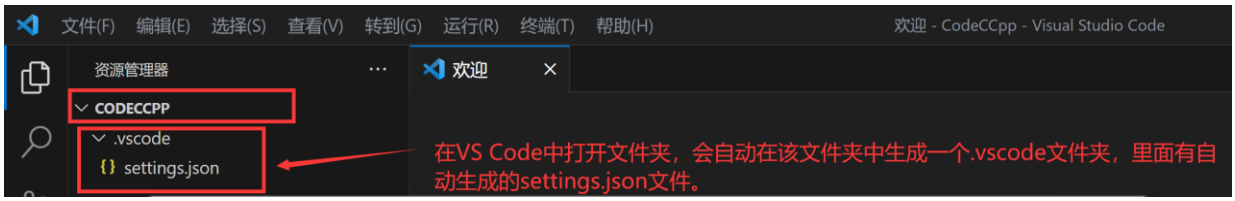
（1） 创建和打开文件夹

在电脑上新建一个文件夹专门用于存放自己编写的 C 语言和 C++语言程序，如 D:\CodeCCpp，也可以使用其他文件夹名称，文件夹名称必须全英文。

启动 VS Code，点击“文件”菜单选择“打开文件夹”，或者点击启动界面中的“打开文件夹”，选择打开自己建好的文件夹“CodeCCpp”，如图所示。

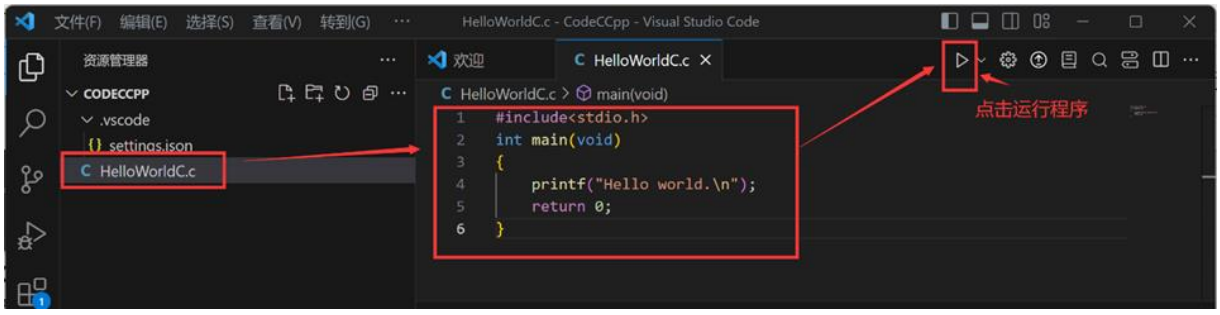


在 VS Code 中打开文件夹后，左侧“资源管理器”中显示打开的文件夹“CODECCPP”(CodeCCpp 自动变为全部大写)，并自动生成文件夹“.vscode”,内含自动生成的文件“settings.json”。

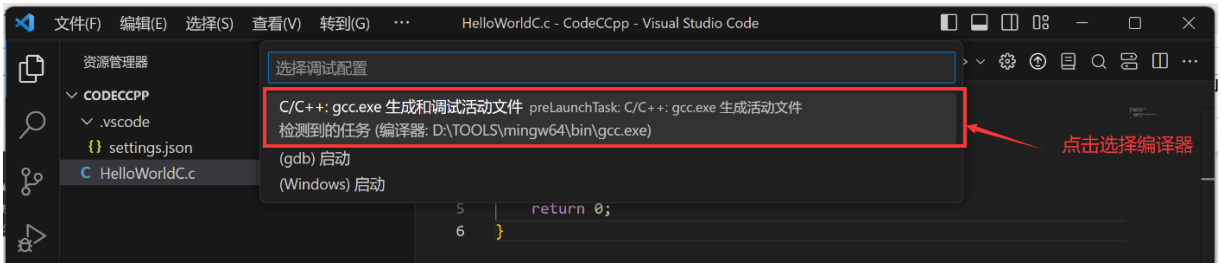


（2）编辑、编译、运行 C 语言源程序

在文件夹中新建一个 C 语言源程序文件，如 HelloWorldC.c。编写 C 语言程序（编辑）如图所示，然后点击右上方工具按钮右侧下拉列表，选择“运行 C/C++文件”，如下图所示。

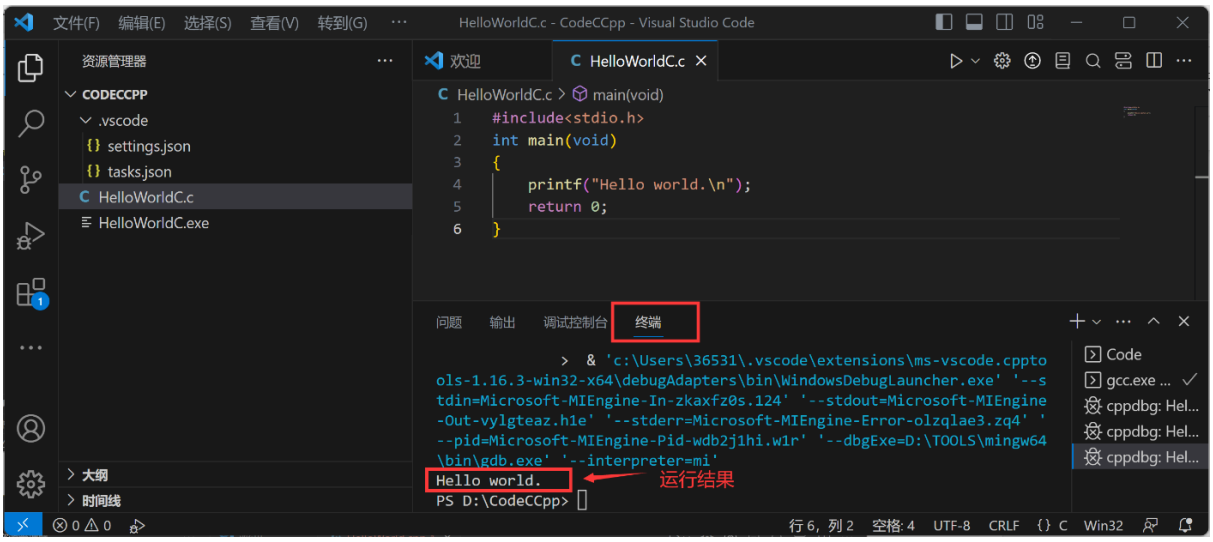


点击“运行 C/C++文件”后，会弹出选择调试配置，点击图示选项，选择 gcc.exe 编译器。

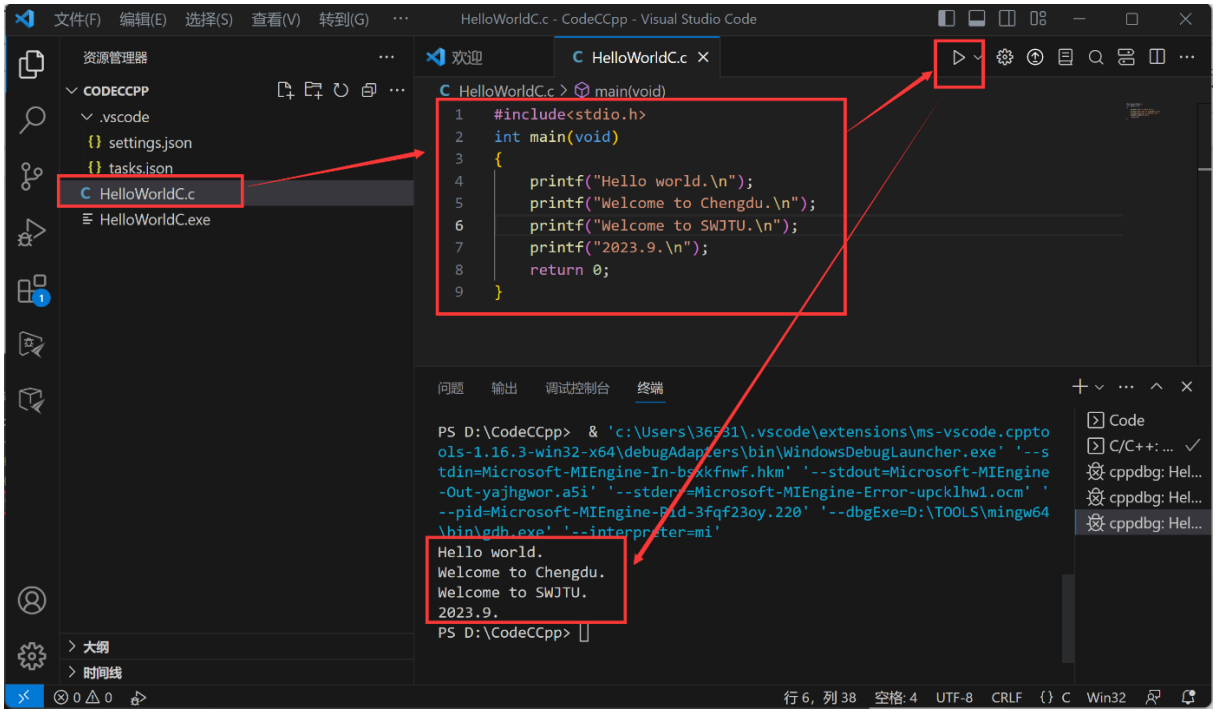


此时，有可能出现错误提示：无法打开源文件“stdio.h”。解决方法：点击报错的代码，点击编辑“includePath”设置下方的“禁用错误波形曲线”即可。

然后，在“终端”就可以看到运行结果。同时，文件夹“.vscode”内自动生成了文件“tasks.json”，文件夹”CODECCPP”内出现可执行文件“HelloWorldC.exe”，如图所示。

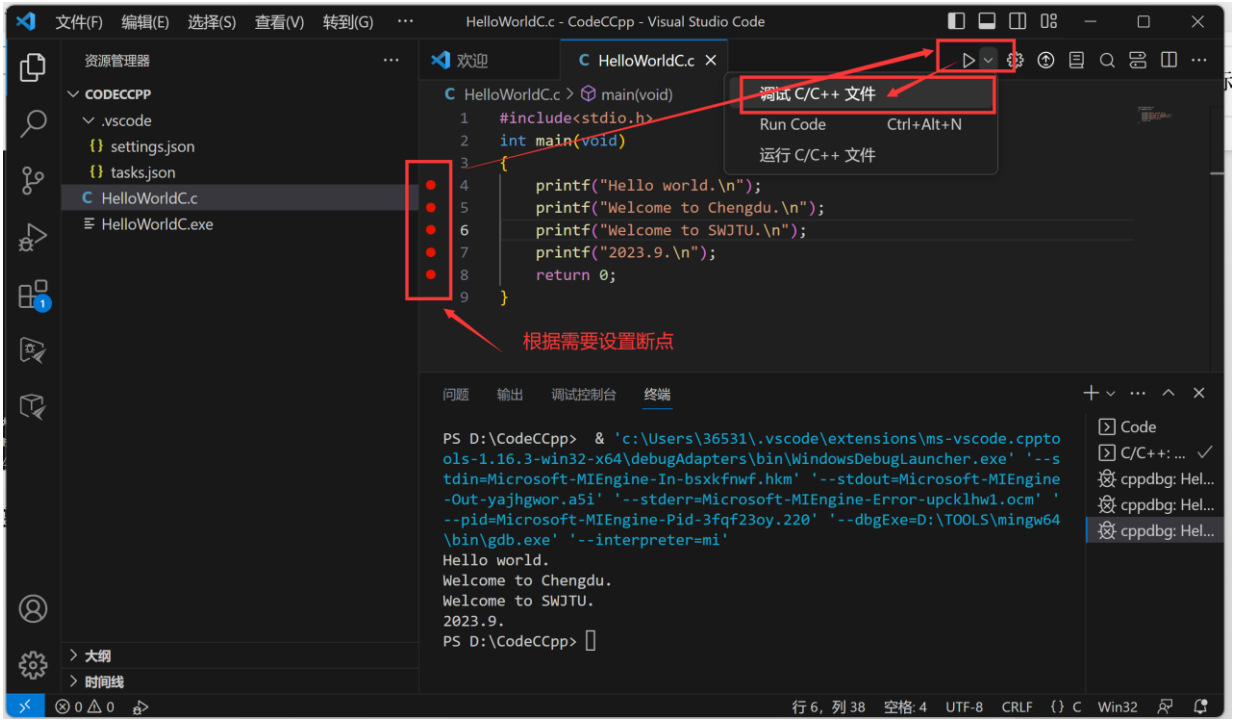


修改 HelloWorldC.c 程序，增加几行代码，点击运行可以看到全部运行结果。

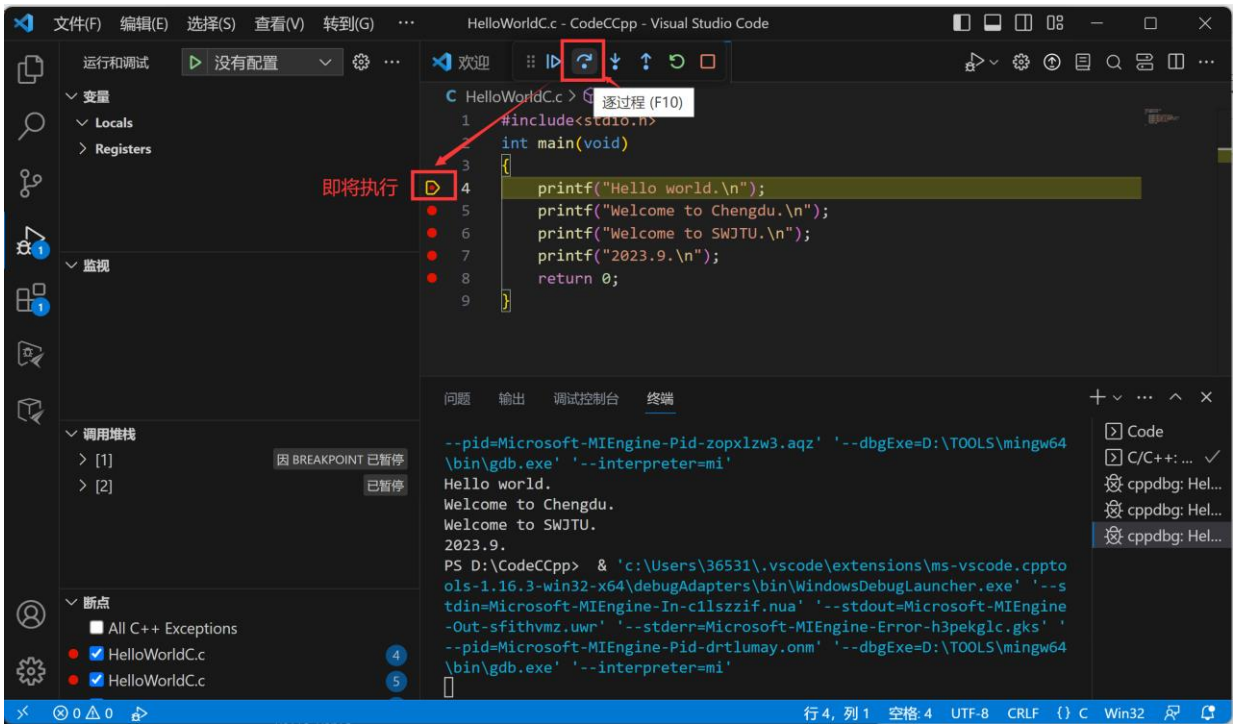


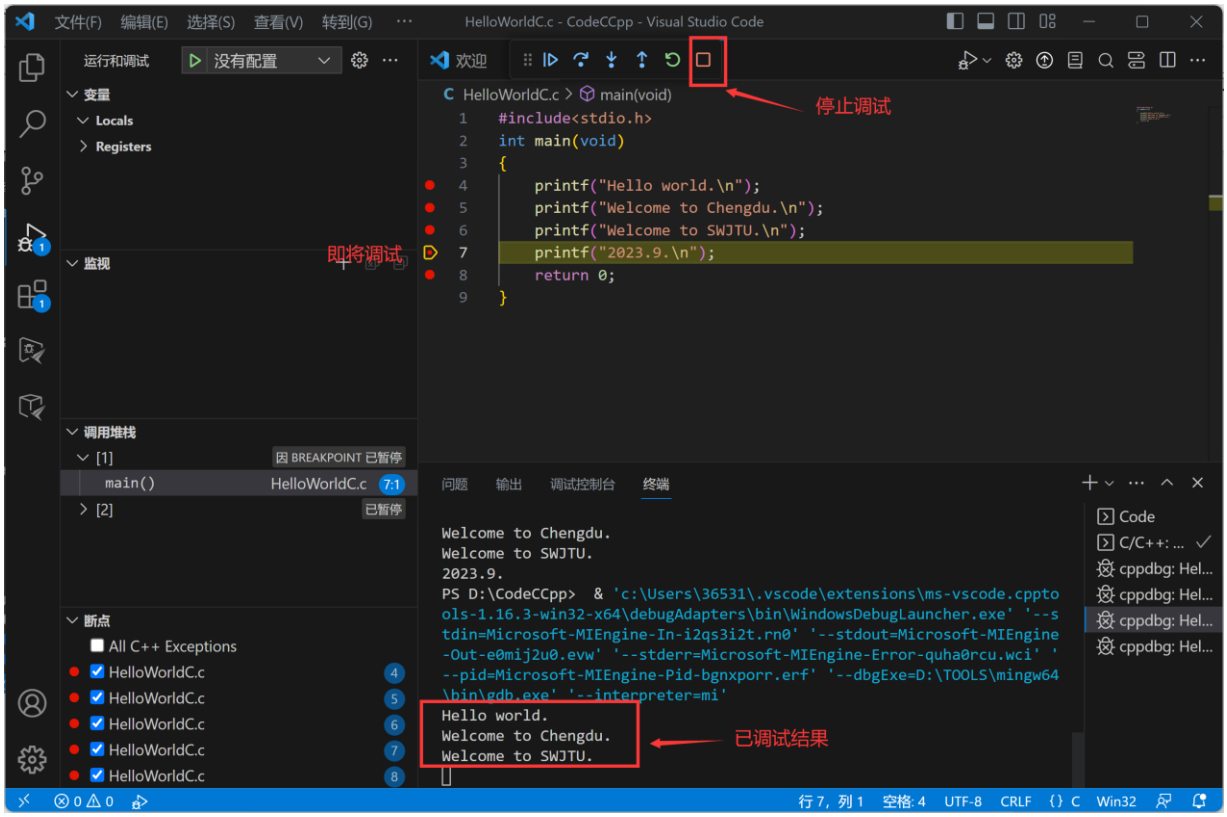
（3）调试 C 语言源程序

VS Code 可以直接运行多行的 C 语言程序，也可以设置断点进行调试，以便程序员了解程序执行过程，发现和修改程序中的错误。如果希望对程序进行调试，可以先根据需要点击程序行号左侧设置断点，选择“调试 C/C++文件”。断点设置后可以再次点击取消断点。



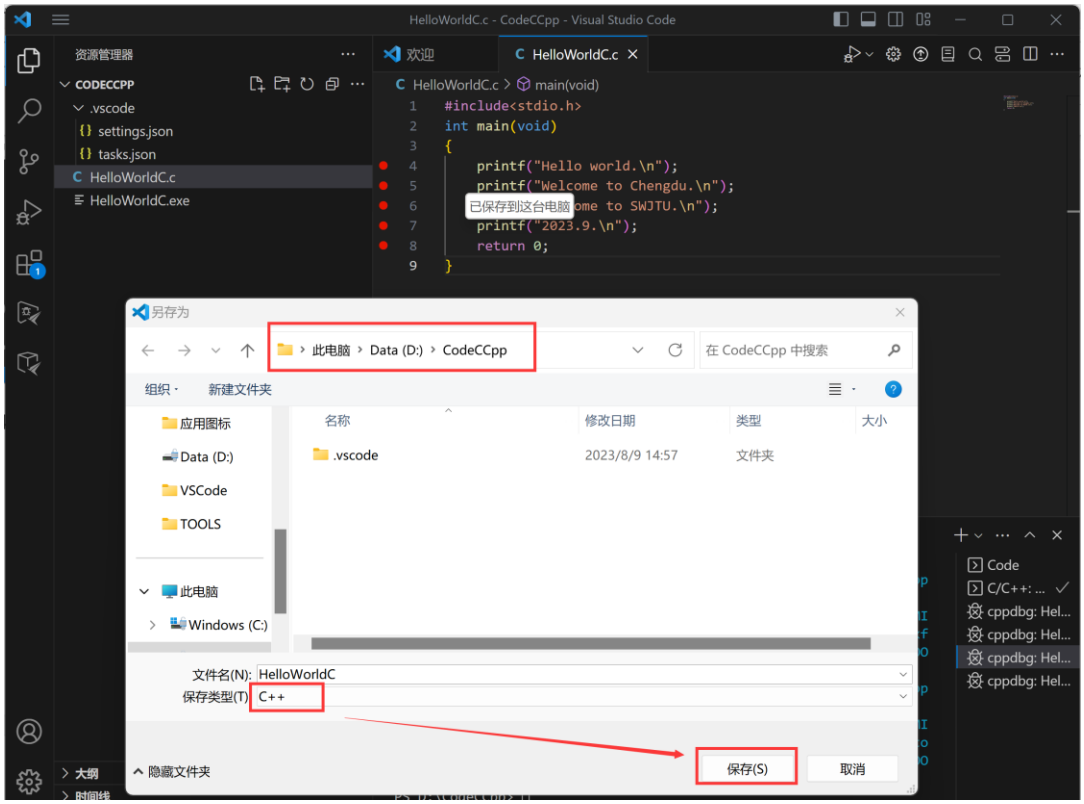
选择“调试 C/C++文件”之后，出现调试工具栏，可以“逐过程调试(F10)”或者“单步调试(F11)”，中途可以“停止调试（Shift+F5）”，也可以“重启调试（Ctrl+Shift+F5)”等。



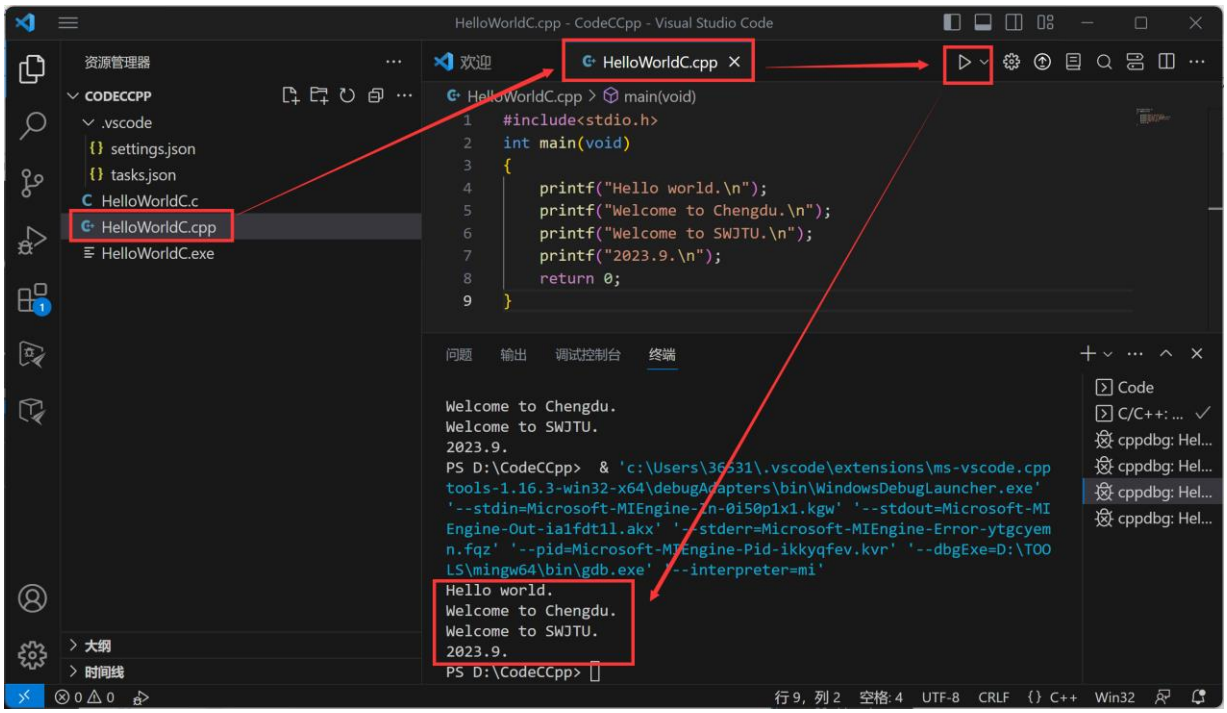


（4）运行扩展名为.cpp 的 C 语言源程序

由于 C++语言兼容 C 语言，目前，大部分情况下都将 C 语言源程序保存为扩展名为.cpp 的源文件。下面我们直接将前面编写的源文件“HelloWorldC.c”，另存为“HelloWorldC.cpp”，如下图所示。



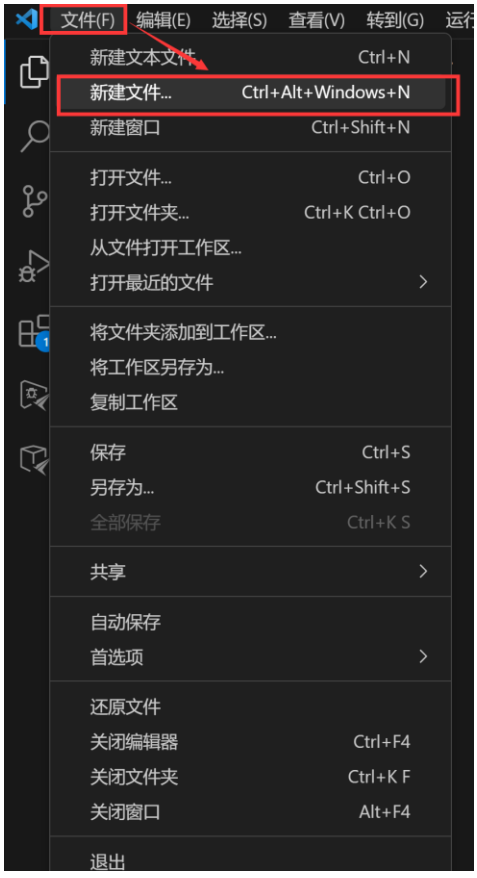
点击“运行”，可以看到“HelloWorldC.cpp”与“HelloWorldC.c”运行结果完全相同。

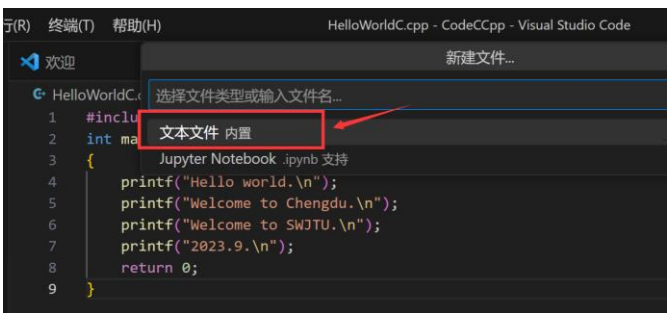


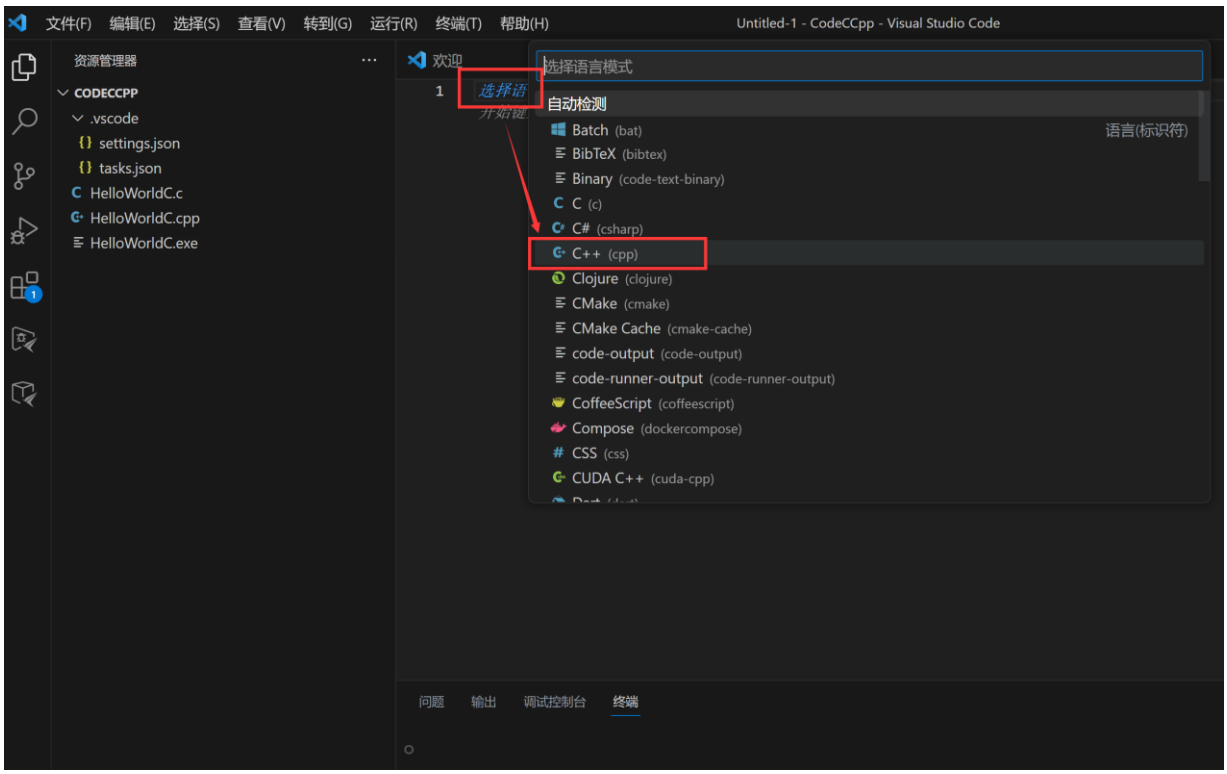
说明：我们编 C 语言程序时也可以直接“新建文件”选择保存类型为“C++”，即可保存为扩展名为.cpp 的 C 语言源程序文件。运行和调试方法与.c 文件相同。

（5）编辑、编译、运行扩展名为.cpp 的 C++语言源程序

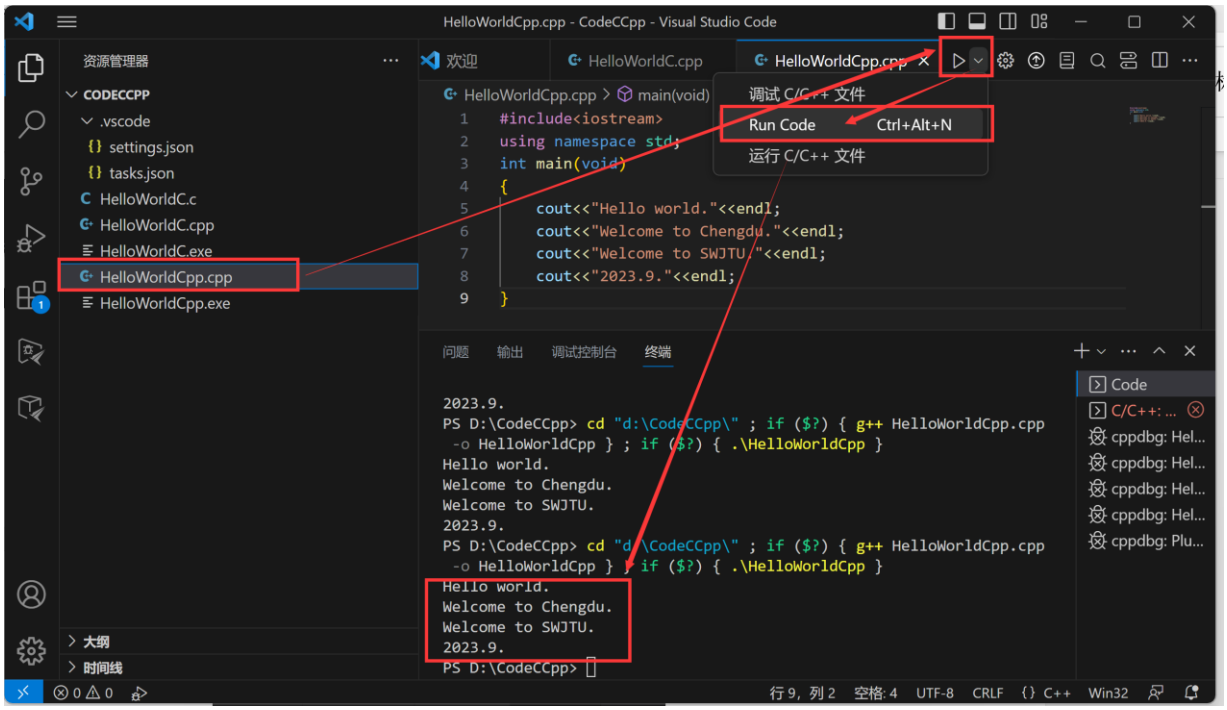
在 VS Code 中选择“文件”菜单，选择“新建文件”，点击“文本文件”，然后“选择语言”“C++”。



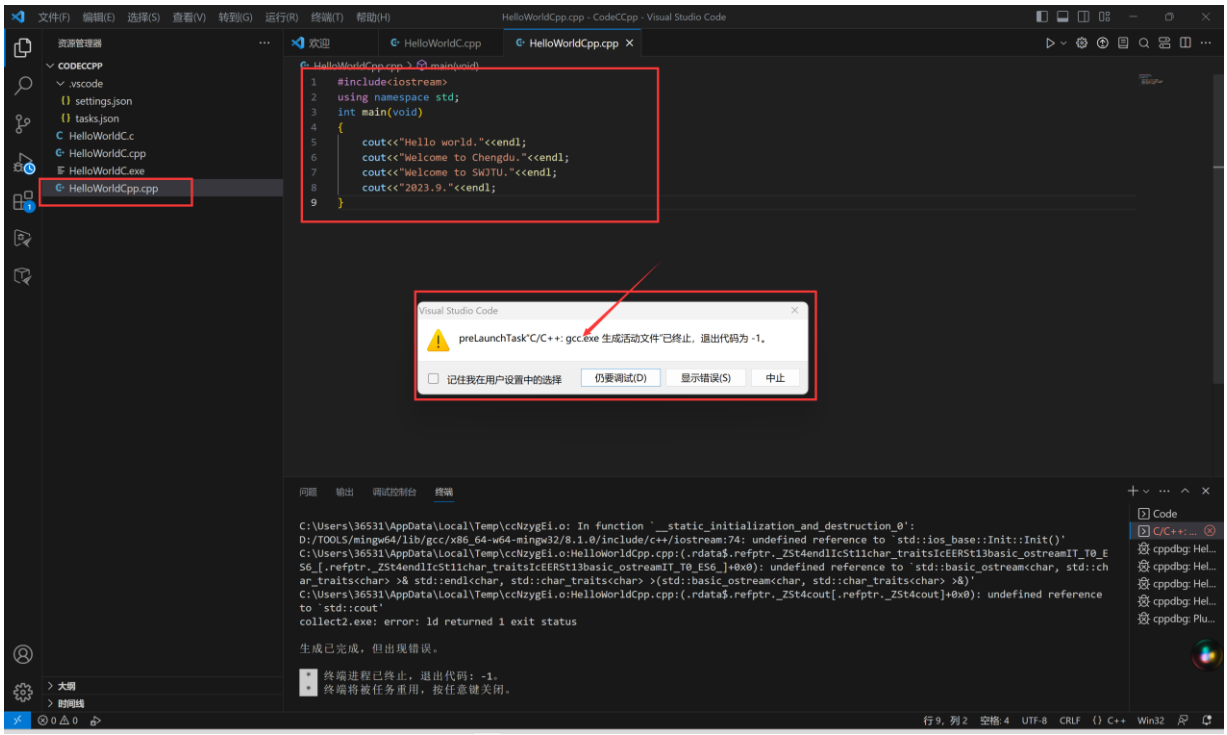




将新建的 Untitle1 另存为”HelloWorldCpp.cpp”。输入 C++语言程序代码，点击“Run Code”即可在终端看到运行结果。至此，已经可以使用 VS Code 运行 C++语言程序。



如果需要调试 C++语言程序，不是点击“Run Code”而是点击“运行 C/C++文件”或者“调试 C/C++文件”，将出现如下图所示错误。这是因为前述运行的程序不管扩展名是“.c”还是“.cpp”都是 C 语言程序，选择使用的编译器是“gcc.exe”,如果要编译调试 C++语言程序，需要将编译器修改为“g++.exe”，调试器设置为“gdb”。



点击错误提示对话框中的“中止”关闭对话框。

（1） 创建 lauch.json 文件

点击“运行”菜单，选择“添加配置”。



在弹出的“选择调试器”列表中选择“C++（GDB/LLDB）”。

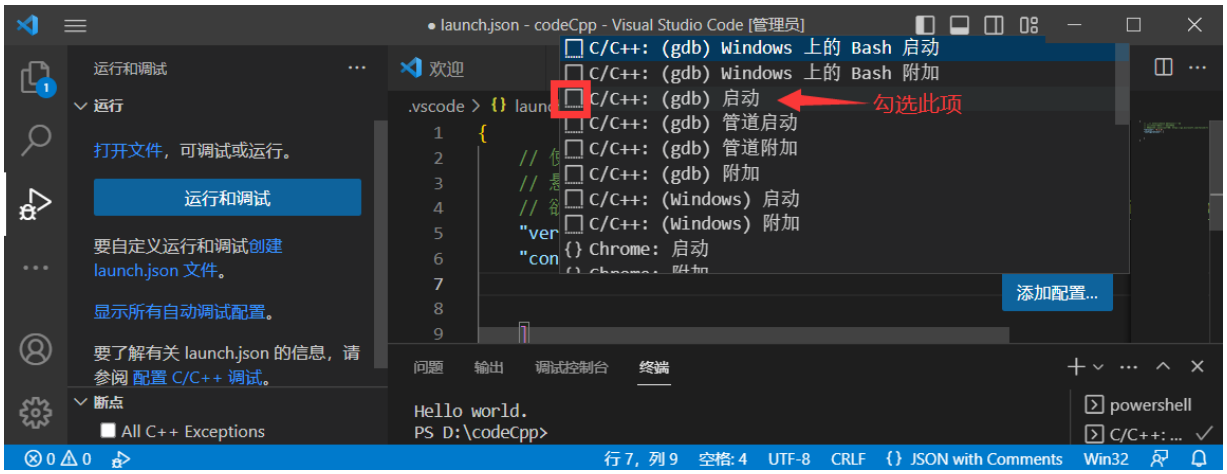


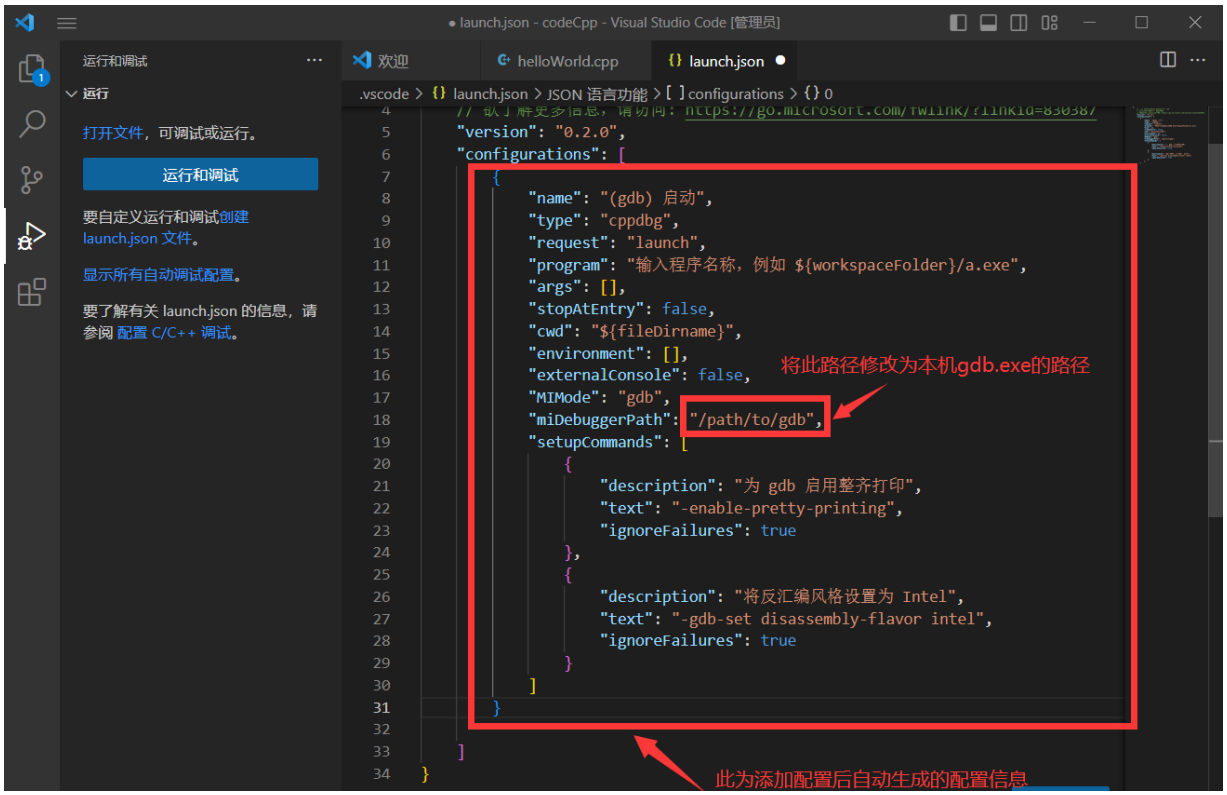
此时会在资源管理器中自动生成文件“launch.json”，但该文件只有版本信息，配置信息还是空的。



再次点击“运行”菜单，选择“添加配置”，可以看到可以选择的配置列表。

勾选：C/C++:(gdb)启动。





将勾选配置后自动生成的配置信息中的路径修改为本机 gdb.exe 文件的路径即配置调试器完成。

如：

```
"D:\\TOOLS\\mingw64\\bin\\gdb.exe"
```

也可不选添加配置，直接复制如下代码到 launch.json 文件中替换掉原有代码，再修改路径。

```
{
    // 使用 IntelliSense 了解相关属性。
    // 悬停以查看现有属性的描述。
    // 欲了解更多信息，请访问: https://go.microsoft.com/fwlink/?linkid=830387
    "version": "0.2.0",
    "configurations": [
        {
            "name": "(gdb) 启动",
            "type": "cppdbg",
            "request": "launch",
            "program": "输入程序名称，例如 ${workspaceFolder}/a.exe",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${fileDirname}",
            "environment": [],
            "externalConsole": false,
            "MIMode": "gdb",
            "miDebuggerPath": "D:\\TOOLS\\mingw64\\bin\\gdb.exe",
            "setupCommands": [
                {
                    "description": "为 gdb 启用整齐打印",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                },
                {
                    "description": "将反汇编风格设置为 Intel",
                    "text": "-gdb-set disassembly-flavor intel",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

（2） 创建 tasks.json 文件

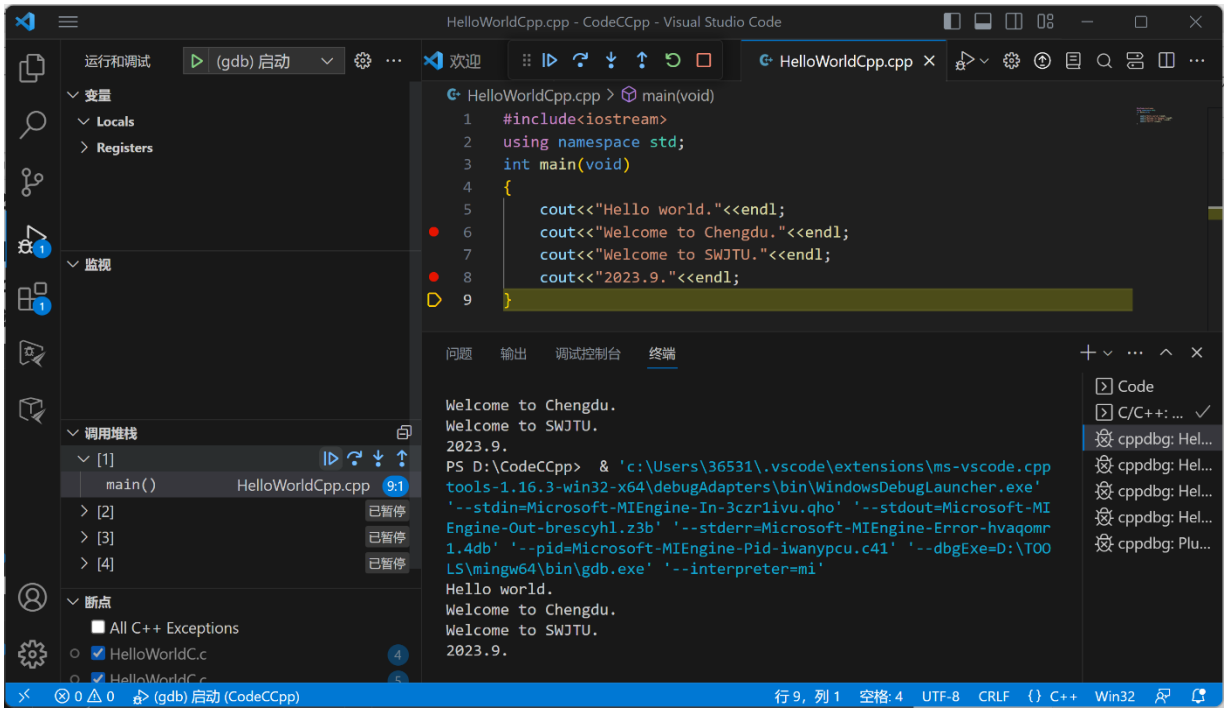
如果已经按照前述步骤新建过 C 语言程序，此时 tasks.json 文件已经存在，且已经自动配置好 gcc.exe 的路径。



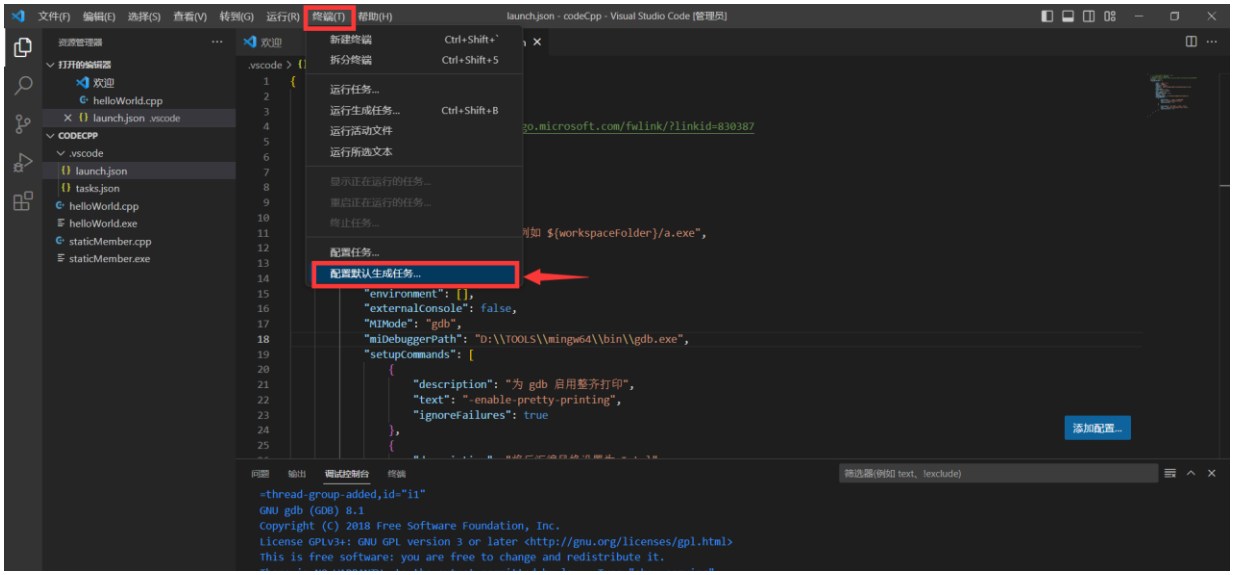
我们只需要将 gcc.exe 修改为 g++.exe，tasks.json 文件就创建并配置成功。



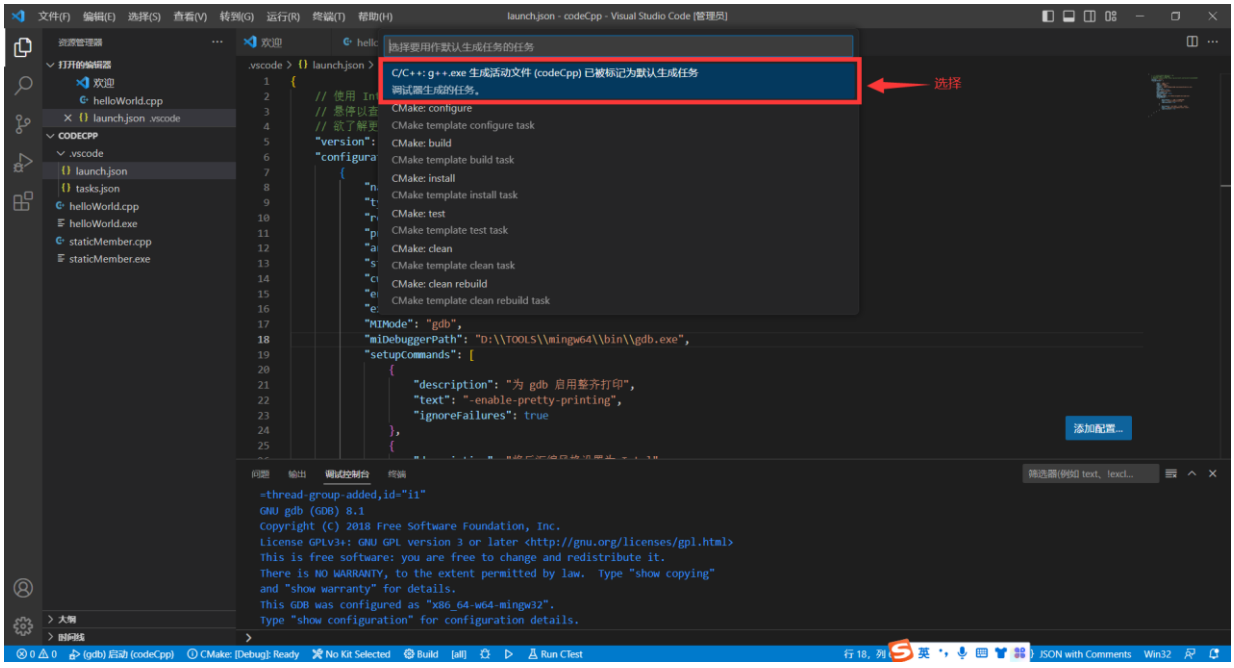
现在就可以运行或者调试 C++语言程序了，如图所示。



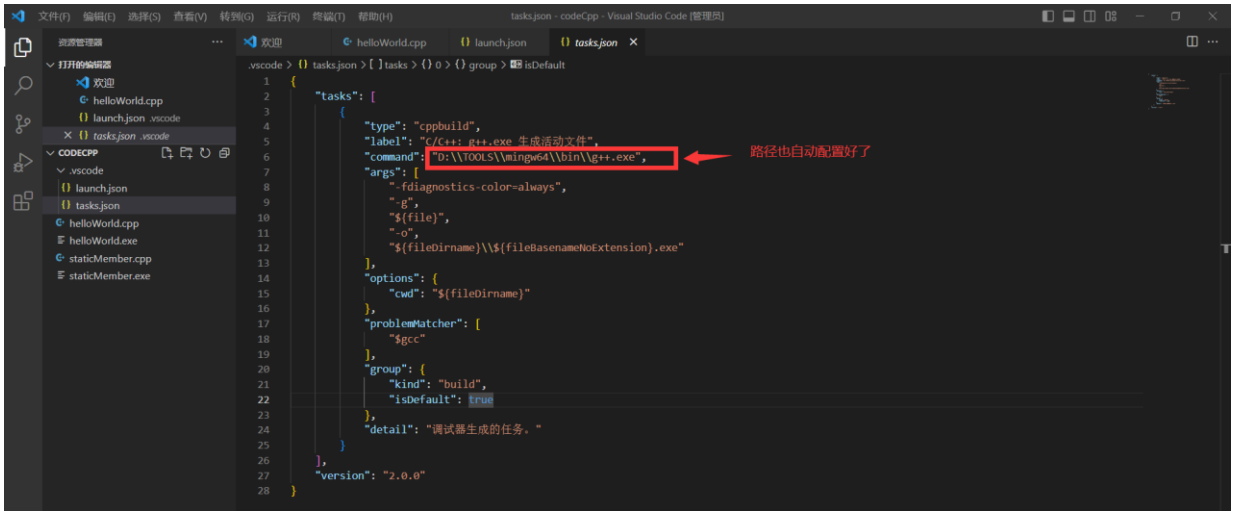
如果没有先运行和调试 C 语言程序，是直接新建的 C++语言源程序文件，可以通过以下步骤创建 tasks.json 文件。首先，在“终端”菜单选择“配置默认生成任务”。



选择“C/C++:g++.exe 生成活动文件……”后自动生成 task.json 文件。



自动生成的 task.json 文件中的路径会自动参照 launch.json 自动生成，无需再修改。



如果要支持 C++11 标准，可在 task.json 文件参数表中添加参数"-std=c++11",如下图所示。

